

RESEARCH ARTICLE

WILEY

Targeted multiobjective Dijkstra algorithm

Pedro Maristany de las Casas¹ | Luitgard Kraus¹ | Antonio Sedeño-Noda² | Ralf Borndörfer¹¹Network Optimization, Zuse Institute Berlin (ZIB), Berlin, Germany²Departamento de Matemáticas, Estadística e Investigación Operativa, Universidad de La Laguna, San Cristóbal de La Laguna, Spain

Correspondence

Pedro Maristany de las Casas, Network Optimization, Zuse Institute Berlin (ZIB), Berlin, 14195, Germany.

Email: maristany@zib.de

Funding information

Bundesministerium für Bildung und Forschung, Grant/Award Number: 05M20ZBM; Ministerio de Ciencia, Innovación y Universidades, Grant/Award Number: PID2019-104928RB-I00.

We introduce the Targeted Multiobjective Dijkstra Algorithm (T-MDA), a label setting algorithm for the One-to-One Multiobjective Shortest Path (MOSP) Problem. It is based on the recently published Multiobjective Dijkstra Algorithm (MDA) and equips it with A*-like techniques. For any explored subpath, a label setting MOSP algorithm decides whether the subpath can be discarded or must be stored as part of the output. A major design choice is how to store subpaths from the moment they are first explored until the mentioned final decision can be made. The T-MDA combines the polynomially bounded size of the priority queue used in the MDA and a *lazy management* of paths that are not in the queue. The running time bounds from the MDA remain valid. In practice, the T-MDA outperforms known algorithms from the literature and the increased memory consumption is negligible. In this paper, we benchmark the T-MDA against an improved version of the state of the art NAMOA_{dr}* One-to-One MOSP algorithm from the literature on a standard testbed.

KEYWORDS

A*, dynamic programming, heuristic search, label setting, multiobjective Dijkstra, multiobjective shortest path

1 | INTRODUCTION

The Multiobjective Shortest Path (MOSP) Problem is an extension of the classical Shortest Path Problem in which every arc bears a multidimensional vector of cost attributes. The multidimensional costs of a path are then the sum of the cost vectors along its arcs. If two paths p and q connect the same end nodes in the graph, p *dominates* q if the cost vector of p is not worse than the cost vector of q in any dimension and it is better than the cost vector of q in at least one dimension. A path is called *efficient*, if it is not dominated by any other path connecting the same end nodes. The cost vector of an efficient path is called a *non-dominated* cost vector. In our setting, the MOSP problem is to find a representative efficient path for every non-dominated cost vector. In the literature this is known as the task of finding a *minimal complete set of efficient paths*. This stands in contrast to the task of finding a *maximal complete set of efficient paths*, where every representative efficient path for a non-dominated cost vector has to be computed.

Definition 1 (One-to-One Multiobjective Shortest Path Problem). Given a digraph $G = (V, A)$, a source node $s \in V$, a target node $t \in V$ and d -dimensional arc costs vectors $c_a \in \mathbb{R}_{\geq}^d$, $a \in A$, the d -dimensional One-to-One Multiobjective Shortest Path (MOSP) problem is to find a minimal complete set of efficient s - t -paths. We denote a d -dimensional One-to-One MOSP instance by (G, s, t, d, c) .

Even in our scenario the number of paths in the problems' output can be exponential w.r.t. the instance size. Despite this principal intractability, MOSP problems arising from real world scenarios can often be solved efficiently. Indeed, significant advances in this respect have been made in recent years. Sedeño-Noda and Colebrook [21] introduced the Biobjective Dijkstra

Algorithm (BDA) for the Biobjective Shortest Path (BOSP) problem that improved the known complexity bounds for this problem and is also efficient in practice. Maristany et al. [14] introduced the Multiobjective Dijkstra Algorithm (MDA), which generalizes the BDA to the multidimensional case. Its running time and space complexity bounds improve the state of the art in theory and practice.

The A* label setting algorithm for One-to-One Shortest Path problems [8,9] gets a *node potential* as part of its input. These potentials, often called *heuristics* in the literature, underestimate the optimal costs from a node to the target node. Then, every path is evaluated based on the sum of its exact costs and the heuristic value associated to the path's end node. Hence, heuristics alter the priority with which paths are processed, benefiting paths with good heuristic values, that is, paths that are expected to reach the target along a cheap extension. Mandow and Pérez de la Cruz [10,11] generalized this notion to the multiobjective scenario. Later they introduced the NAMOA* algorithm [17]. An improved version called NAMOA*_{dr} algorithm [18], is still the state of the art One-to-One MOSP algorithm in the literature.

Consider a path p analyzed by a label setting MOSP algorithm. During the iterations of the algorithm that elapse from the time point in which p is seen for the first time until it is either expanded along the outgoing arcs of its end node or discarded forever, p is considered an *explored* path. An important design choice when it comes to develop MOSP algorithms is how to handle explored paths efficiently. Three ways of doing so appear in the literature. To describe them, we consider a One-to-One MOSP instance (G, s, t, d, c) as in Definition 1. All algorithms use a priority queue to store (some of the) explored paths.

- (I) For any node $v \in V$, the MDA [14] and BDA [21] store at most one explored s - v -path p at a time in the algorithms' priority queue. No other explored paths are stored neither in the priority queue nor in other containers. Explored paths that were seen but did not match the criteria to enter the queue are rebuilt as needed exploiting the subpath efficiency of efficient paths (cf. [15] [Lemma 4]). In this approach the number of explored paths that are stored simultaneously is minimal.
- (II) The classical label setting algorithm by Martins [15] stores more explored paths. For every node $v \in V$ a list of explored s - v -paths exists. No two elements in one of these lists dominate each other. Maintaining this property is achieved using so called *merge operations*. These operations have been proven to be costly in theory and practice [13,14,21]. Efficient implementations of Martins' algorithm store only a minimal (w.r.t. the chosen ordering of paths) path from every such list in the algorithm's priority queue.
- (III) Recently, Ulloa et al. [23] and Böckler [4] proposed *lazy queue management* for One-to-One BOSP algorithms. The name (*lazy*) is often used in the literature and is motivated by the fact that new explored paths are added to the algorithms' priority queue already if they are not dominated by already found efficient paths with the same end nodes. It is neither checked if existing explored paths dominate them nor if the new paths dominate existing explored paths. In consequence, dominance checks on every path need to be repeated after their extraction from the priority queue. The practical advantage is that the handling of every explored path is easy and does not require any additional data structures besides the algorithms' priority queue. The theoretical drawback of this approach is that an exponential number of irrelevant paths may be in the queue simultaneously. That is, a path in the queue can dominate another path with the same end nodes that is simultaneously in the queue.

On a standard testbed of instances containing road networks of the United States with integer costs, Ahmadi et al. [1] show that the theoretical drawback of lazy queue management (cf. item 1 in the above enumeration) is not a concern and their algorithm constitutes the state of the art on these instances. The upcoming question is how the lazy queue management technique performs in the general multiobjective scenario. This question is meaningful because for more than two objectives, the complexity of MOSP problems increases w.r.t. the biobjective case [14] making it impossible to extrapolate the biobjective behavior to higher dimensions.

Our first and main contribution is the *Targeted Multiobjective Dijkstra* algorithm (T-MDA). It is based on the MDA introduced in [14] since it stores at most one explored s - v -path in its priority queue for every node $v \in V$ (cf. item 1 in the above enumeration). This ensures that queue operations remain efficient in theory and practice. The T-MDA however also resembles lazy algorithms: every explored s - v -path p that is not inserted into the queue is not discarded and rebuilt later. Instead, it is stored in a list of explored s - v -paths associated to the last arc, say $(u, v) \in A$ of p . This list contains all relevant explored s - v -paths whose last arc is (u, v) . If the priority queue of the algorithm is sorted in lexicographically (cf. Definition 2), only insertions of paths at the beginning or at the end of these lists (i.e., constant time insertions) are needed to ensure that they also remain lexicographically sorted. Hence, the new T-MDA combines the polynomially sized priority queue of explored paths used in the MDA with a lazy management of the remaining explored paths to avoid having to rebuild paths multiple times.

The state of the art algorithm for One-to-One MOSP problems is the NAMOA* algorithm introduced in [18]. It combines Martins' algorithm with A* techniques from the literature. In particular it handles explored paths using merge operations as described in Item 1 of the above enumeration. Our second contribution is the design of the NAMOA*_{dr}-lazy algorithm, a variant of the NAMOA*_{dr} algorithm that uses lazy queue management as described in Item 1 of the above enumeration.

To the best of our knowledge such an algorithm is novel in the literature and generalizes the ideas from [1,23] to the multiobjective case.

The paper is structured as follows. In Section 2, we formalize the notation used in the paper and discuss the *dimensionality reduction technique* to check dominance more efficiently. This technique is applicable to all algorithms considered in the paper. In Section 3, we discuss how to compute heuristics for any given One-to-One MOSP instance in polynomial time. These heuristics are needed to redefine the arc costs of the original problem instance to guide the search towards the target node. We then proceed to describe the T-MDA algorithm, prove its correctness, and analyze its theoretical space and running time complexity bounds in Section 4. Furthermore, in Section 5, we briefly discuss the NAMOA_{dr}^{*} algorithm and how to design a *lazy* version of the algorithm. Finally, in Section 6, we report on extensive computational results on three dimensional One-to-One MOSP instances on different types of graphs.

2 | LEXICOGRAPHIC ORDER, DOMINANCE, AND EFFICIENCY

In this section we introduce the operations and notations needed in the remainder of the paper. Most concepts are used in different multiobjective combinatorial optimization problems and their definitions vary slightly depending on the source.

Definition 2 (Lexicographic Ordering). Let $x, y \in \mathbb{R}^d$ be two d -dimensional vectors, $d \in \mathbb{N}$. x is lexicographically smaller than y if $x_i < y_i$ for the first dimension $i \in \{1, \dots, d\}$ for which $x_i \neq y_i$. In this case, we write $x <_{lex} y$. Given this definition, we use the binary relation $\leq_{lex}$ on $\mathbb{R}^d$ canonically.

Recall that the binary relations $<$ and $\leq$ are canonically defined componentwise on vectors.

Definition 3 (Dominance). Let $x, y \in \mathbb{R}^d$ be two d -dimensional vectors, $d \in \mathbb{N}$. x dominates y if $x \leq y$ and $x \neq y$. That is, $x_j < y_j$ in at least one index $j \in \{1, \dots, d\}$. Given a set $X \subset \mathbb{R}^d$, $x \in X$ is said to be a non-dominated vector in X if it is not dominated by any other vector in X .

Definition 4 ((Lex. Sorted) Front). Let $X \subset \mathbb{R}^d$ be a discrete set, $d \in \mathbb{N}$. X is called a d -dimensional front if it is sorted in lex. non-decreasing order and no two vectors in X dominate each other.

In this paper, we apply the operator $\leq_D$ to check if a vector is contained or is dominated by at least one vector in a given set. This operation is the key for building minimal complete sets of efficient paths iteratively during the discussed algorithms.

Definition 5 (Dominance-or-Equivalence Operator $\leq_D$). Let X be a front and $x \in \mathbb{R}^d$ a vector. We write $X \leq_D x$ if and only if there is at least one vector in X that dominates x or if x is contained in X .

It is easy to see that $\leq_D$ requires $\mathcal{O}(d|X|)$ comparisons. Moreover, efficient implementations exploit the lex. ordering of fronts to stop the search as soon as an element in X is lex. greater than the input vector x . If that is the case, x is not dominated by any element in X because vectors that are lex. greater than x cannot dominate it.

Definition 6 (Merge Operation). Given a d -dimensional front X and a vector $x \in \mathbb{R}^d$, a merge operation has three purposes:

- 1 Answer whether $X \leq_D x$ and if not,
- 2 insert x into X s.t. X remains sorted in lex. non-decreasing order and finally
- 3 remove elements in X that are dominated by x .

A merge operation requires $\mathcal{O}(d|X|)$ comparisons if the front X is implemented as a list that allows constant time insertion and removal of elements in random positions. Again, the facts that fronts are lex. sorted by definition and that a vector can only dominate lex. greater vectors must be exploited.

Definition 7 (Dimensionality Reduced Front. [18]). Consider the set of $(d - 1)$ -dimensional vectors that arises after neglecting the first dimension of every vector in a d -dimensional front X . The dimensionality reduced front of X is the $(d - 1)$ -dimensional front $X_{dr} \subset \mathbb{R}^{d-1}$ that arises after restoring the front properties in this set.

Example. Consider the set $X = \{(1, 2, 3), (3, 2, 1), (2, 1, 3)\}$. To build X_{dr} , the 2-dimensional vectors $\{(2, 3), (2, 1), (1, 3)\}$ obtained after neglecting the first component of the original vectors are considered. Since $(2, 3)$ is dominated by the other vectors in this set, it is discarded. Finally after re-sorting the remaining vectors, we obtain $X_{dr} = \{(1, 3), (2, 1)\}$.

Note that by construction, $|X_{dr}| \leq |X|$. With this in mind and since vectors in X_{dr} are lower dimensional than vectors in X , the following theorem motivates the use of dimensionality reduced fronts to speedup dominance checks.

Theorem 1 (cf. Lemma 2 in [18]). Consider a vector x that is not lex. smaller than any vector in X . Then,

$$X \leq_D x \Leftrightarrow X_{dr} \leq_D (x_2, \dots, x_d).$$

Consider a d -dimensional One-to-One MOSP instance (G, s, t, d, c) , $d \in \mathbb{N}_{\geq 2}$. The costs $c(p)$ of a path p are defined as the sum of its arc costs, that is, $c(p) := \sum_{a \in p} c_a$. Moreover, we set the dimensionality reduced costs of p to be $c_{dr}(p) := (c_2(p), \dots, c_d(p)) \in \mathbb{R}^{d-1}$. For a set $\mathcal{P}$ of paths in G with coincident end nodes, $c(\mathcal{P})$ is the set of costs of all paths in $\mathcal{P}$, that is, $c(\mathcal{P}) := \{c(p) \mid p \in \mathcal{P}\} \subset \mathbb{R}_{\geq 0}^d$. Moreover, we refer to the dimensionality reduced front induced by the paths in $\mathcal{P}$ with $c_{dr}(\mathcal{P})$.

Definition 8 (Lex. Order, Dominance and Efficiency of Paths). Let p and q be two paths in G connecting the same end nodes. We define:

- 1 p is lex. smaller than q if $c(p) <_{lex} c(q)$.
- 2 p dominates q if $c(p)$ dominates $c(q)$.
- 3 If there is no path in G that dominates p , p is called *efficient*.

Let $\mathcal{P}$ be the set of all efficient paths in G connecting two given nodes. The set $c(\mathcal{P})$ is unique but for any cost vector therein, multiple efficient paths in $\mathcal{P}$ can exist. In this paper, we want to find a *minimal complete set* of efficient paths, that is, a subset of $\mathcal{P}$ that contains exactly one path for every non-dominated cost vector in $c(\mathcal{P})$. To achieve this, the $\leq_D$ operator is used to check if candidate paths are dominated by already found paths or a path with an equivalent cost vector has already been found.

3 | HEURISTICS AND DOMINANCE BOUNDS

The following definitions and results are a generalization of those valid for the single criteria Shortest Path Problem. Throughout this section, we assume that a d -dimensional One-to-One MOSP instance $\mathcal{I} = (G, s, t, d, c)$ is given. *Heuristics* are needed to guide MOSP algorithms towards the target, that is, to identify and prioritize promising s - t -subpaths early. *Dominance bounds* are cost vectors that do not dominate any efficient s - t -path. Thus, (sub-)paths that are dominated by these bounds can be neglected.

Definition 9 ((Admissible and Monotone) Heuristic. cf. [17]). A node potential $\pi : V \mapsto \mathbb{R}_{\geq}^d$ is called an *admissible and monotone heuristic* for $\mathcal{I}$, if

Admissibility:

- i $\pi(v) \leq c(p)$, $\forall v \in V$, p efficient v - t -path,

Monotonicity:

- i $\pi(t) = 0$, and
- ii $\pi(u) \leq c_{uv} + \pi(v)$, $\forall (u, v) \in A$.

Remark. As in the single objective case, a monotone heuristic is always admissible.

Throughout the paper, we only consider monotone and admissible heuristics. For the sake of brevity, we refer to them just as *heuristics* from now on. The notation π to refer to the heuristics is motivated by the dual linear program of the single criteria Shortest Path Problem. There, the set of feasible solutions corresponds to the set of monotone heuristics, often denoted by π . According to the notation from this field, the *reduced costs* of an arc $(u, v) \in A$ are $\bar{c}_{uv} := c_{uv} + \pi(v) - \pi(u)$ and we set

$$\bar{c}(p) := c(p) + \pi(v). \quad (1)$$

The following two lemmas are trivial to prove using the properties of heuristics. We need the statements for the correctness proofs of the algorithms in this paper.

Lemma 2. If π is a heuristic for $\mathcal{I}$, we have $\bar{c}_a \geq 0$ for every arc $a \in A$. As a consequence, if node u comes before node v along any simple path p in G starting at s , $\bar{c}(p_{su}) \leq \bar{c}(p_{sv})$ holds.

Deciding whether subpaths can be extended to become efficient s - t -paths is particularly difficult at the beginning of an algorithm since little information about the set of efficient solutions is known a priori. The following definition describes cost vectors that act as a bound. Paths whose cost vector is dominated by this bound are irrelevant.

Definition 10 (Dominance bound). A dominance bound for $\mathcal{I}$ is a vector $\beta_t \in \mathbb{R}^d$ s.t. there is no efficient s - t -path that is dominated by β_t .

Finding good heuristics and dominance bounds is an application dependent task. For example, the methodology to derive them in flight planning on airway networks [3] and electric vehicle routing on road networks [2] varies and requires application

specific techniques. There is however a generic way of computing heuristics and dominance bounds for d -dimensional One-to-One MOSP instances in polynomial time. This method is often used in the literature [1,7,14,23] and requires d queries of the Dijkstra's algorithm [6] sorting paths lexicographically. These queries are ran from the target node t to all other nodes (One-to-All Dijkstra) on the reversed digraph of G . Thus, for every node $v \in V$, d cost vectors corresponding to v - t -paths in G are obtained and their entries are used to build heuristics and dominance bounds. A detailed exposition of this generic method to obtain heuristics can be found in [7] or in [14].

In Section 4, we assume that a heuristic and a dominance bound are part of the input of the described algorithm.

4 | THE TARGETED MULTIOBJECTIVE DIJKSTRA ALGORITHM

In this section, we introduce and analyze the *Targeted Multiobjective Dijkstra Algorithm* (T-MDA). The pseudocode is shown in Algorithm 1. The T-MDA is a label-setting One-to-One MOSP algorithm. Its input is a d -dimensional MOSP instance (G, s, t, d, c) , a heuristic π , and a dominance bound β_t . The output is a minimum complete set of efficient s - t -paths $\mathcal{P}_{st}$. In practice, paths are stored as *labels* [1,14,21], a memory efficient implicit representation of paths that avoids storing efficient subpaths multiple times. Thus, the algorithm stores lists $\mathcal{P}_{sv}$, $v \in V$, of efficient s - v -paths. When implementing the algorithm, these paths correspond to stored labels that are used to reconstruct the efficient s - t -paths when the T-MDA terminates. Paths that are added to these lists are called *permanent*. Permanent paths are efficient subpaths but not every efficient s - v -path p is made permanent: if all extensions of p along the outgoing arcs of v are dominated subpaths, p is discarded since it is not part of any efficient s - t -path. This follows from the fact that subpaths of efficient s - t -paths are efficient (cf. [15] [Lemma 4]). Additionally, for every $v \in V$, the algorithm stores the dimensionality reduced front $\bar{c}_{dr}(\mathcal{P}_{sv})$ of non-dominated cost vectors belonging to already found efficient s - v -paths. Using Theorem 1, the fronts $\bar{c}_{dr}(\mathcal{P}_{sv})$ are used to determine if new s - v -paths are dominated.

A path p is an *explored path* if it has been analyzed by a label setting algorithm but it is not yet a permanent path or has been discarded. The T-MDA manages these paths in a novel way. The most promising paths are stored in the algorithm's priority queue. The following definition formalizes this notion.

Definition 11 (Queue paths). Is at most one s - v -path q in the priority queue Q . Consider the efficient s - v - and s - t -paths stored in $\mathcal{P}_{sv}$ and $\mathcal{P}_{st}$ in some iteration of the T-MDA. In this iteration, an s - v -path p is the queue path for v if

- 1 the cost vector $\bar{c}(p)$ is not dominated by or equivalent to any cost vector in $\bar{c}(\mathcal{P}_{sv})$,
- 2 the cost vector $\bar{c}(p)$ is not dominated by or equivalent to any cost vector in $\bar{c}(\mathcal{P}_{st})$, and
- 3 among all known s - v -paths that fulfill conditions 1 and 2, p is lex. minimal w.r.t. $\bar{c}$.

Explored s - v -paths that are not the queue path for v are stored in so called $\mathcal{NQP}$ (next queue path) lists. The T-MDA maintains an $\mathcal{NQP}_a$ list for every arc $a \in A$ and any explored path that is not a queue path is stored in the $\mathcal{NQP}$ list corresponding to the path's last arc. The main invariants of the algorithm are:

- The priority queue Q and the $\mathcal{NQP}$ lists are sorted in lex. non decreasing order w.r.t. $\bar{c}$.
- For any $v \in V$, there is at most one queue path for v in Q (see Definition 11).
- The $\mathcal{NQP}$ lists are maintained in a lazy way, that is, they might contain dominated paths.
- For any $v \in V$, when an s - v -path is extracted from Q , a new queue path for v is picked among the s - v -paths in the lists $\mathcal{NQP}_{uv}$, $u \in \delta^-(v)$. During this search, dominated paths in these lists are discarded.

The data structures of the T-MDA are initialized in Line 1–Line 3 in Algorithm 1. Initially, they are all empty. Before the main loop of the algorithm begins, the trivial path p_{init} from s to itself is inserted into Q (Line 5 in Algorithm 1). The main loop finishes when Q is empty at the beginning of an iteration. Every iteration begins with the extraction of a lex. minimal (w.r.t. $\bar{c}$) path p from Q (Line 7 in Algorithm 1). In Lemma 6, we prove that extracted paths are efficient. The set of extracted s - t -paths build the output of the algorithm. In the remainder of this section, we describe how the T-MDA proceeds in every iteration.

Merge of extracted paths (Line 9 in Algorithm 1).

Assume that the extracted path p in the current iterations is an s - v -path for some $v \in V$. In Lemma 6 we prove that p is efficient and in Theorem 5 we prove that p is lex. greater than all s - v -paths that were extracted in prior iterations. Hence, we use the merge procedure (Definition 6) to update the dimensionality reduced front $\bar{c}_{dr}(\mathcal{P}_{sv})$. Note that since p is guaranteed to be efficient, there is no need to check whether $\bar{c}_{dr}(p)$ is dominated. $\bar{c}_{dr}(p)$ is always added to $\bar{c}_{dr}(\mathcal{P}_{sv})$. Taking this into account while implementing the T-MDA enhances the practical performance of the algorithm. Then, vectors in $\bar{c}_{dr}(\mathcal{P}_{sv})$ that are dominated by $\bar{c}_{dr}(p)$ are deleted. All in all, the cardinality of the reduced front may decrease, achieving faster dominance checks in future iterations.

ALGORITHM 1. Targeted Multiobjective Dijkstra Algorithm

Input : d -dimensional One-to-One MOSP instance $\mathcal{I} = (G, s, t, d, c)$, heuristic π for $\mathcal{I}$, dominance bound β_t for $\mathcal{I}$.

Output: Minimal complete set $\mathcal{P}_{st}$ of efficient s - t -paths.

```

1 Priority queue of paths  $Q \leftarrow \emptyset$ ; // Sorted according to  $\bar{c}$ .
2  $\forall (u, v) \in A$  – explored  $s$ - $v$ -paths that use  $(u, v)$  as last arc:  $\mathcal{NQP}_{uv} \leftarrow \emptyset$ ;
3  $\forall v \in V$  – efficient  $s$ - $v$ -paths:  $\mathcal{P}_{sv} \leftarrow \emptyset$ ;
4  $p_{\text{init}} \leftarrow ()$ ;
5  $Q \leftarrow Q.\text{insert}(p_{\text{init}})$ ;
6 while  $Q \neq \emptyset$  do
7    $p \leftarrow Q.\text{extractMin}()$ ;
8    $v \leftarrow$  last node of path  $p$ ; // If  $p = p_{\text{init}}$ ,  $v \leftarrow s$ .
9    $\bar{c}_{dr}(\mathcal{P}_{sv}) \leftarrow \text{merge}(\bar{c}_{dr}(\mathcal{P}_{sv}), \bar{c}_{dr}(p))$ 
10  success  $\leftarrow$  False;
11  if  $v \neq t$  then  $(Q, \text{success}) \leftarrow \text{propagate}(p, Q, \mathcal{P}, \mathcal{NQP}, \beta_t)$ ;
12  if  $v == t$  or success == True then  $\mathcal{P}_{sv}.\text{append}(p)$ ;
13   $p_v^{\text{new}} \leftarrow \text{nextQueuePath}(p, \mathcal{NQP}, \delta^-(v), \mathcal{P})$ ;
14  if  $p_v^{\text{new}} \neq \text{NULL}$  then  $Q.\text{insert}(p_v^{\text{new}})$ ;
15 return  $\mathcal{P}_{st}$ ;

```

Propagation of extracted paths (Line 11 in Algorithm 1).

The extracted s - v -path p is propagated along the arcs $(v, w) \in \delta^+(v)$. For such an arc (v, w) , we call the resulting path $p_w^{\text{new}} := p \circ (v, w)$ and distinguish three scenarios. To do so, we first need to check if there is an s - w -path in Q already. Since Q contains at most one path per node, this check can be done in $\mathcal{O}(1)$ time using a boolean vector of n entries.

- Case 1: There is no s - w -path in Q . Then, p_w^{new} is inserted into Q and becomes the queue path of w if its reduced costs are not dominated by β_t or by a cost vector in $\bar{c}_{dr}(\mathcal{P}_{sw})$ or in $\bar{c}_{dr}(\mathcal{P}_{st})$.
- Case 2: The s - w -path $q \in Q$ is the current queue path for w and $\bar{c}(p_w^{\text{new}}) <_{\text{lex}} \bar{c}(q)$. Then, if p_w^{new} is not dominated by β_t or any of the vectors in $\bar{c}_{dr}(\mathcal{P}_{sw})$ or in $\bar{c}_{dr}(\mathcal{P}_{st})$ it is inserted via a `decreaseKey` operation into Q , where it replaces q . If q is dominated by p_w^{new} , it can be discarded. Otherwise assume that $(x, w) \in A$ is the last arc of q . Subsequently, q is prepended to the list $\mathcal{NQP}_{xw}$. This means that even though q is no longer in Q , it might re-enter it later.
- Case 3: The s - w -path $q \in Q$ is the current queue path for w and $\bar{c}(q) <_{\text{lex}} \bar{c}(p_w^{\text{new}})$. Then, p_w^{new} is appended to $\mathcal{NQP}_{vw}$ if it is not dominated by q . Note that no dominance checks w.r.t. to the permanent fronts are made in this case. They are postponed and done in the `nextQueuePath` procedure when needed.

The procedure `propagate` (Algorithm 2) returns the possibly updated priority queue Q and a success flag whose value is only set to *True* if one of the considered expansions of p is stored either in Q or in an $\mathcal{NQP}$ list. In this case p is added to $\mathcal{P}_{sv}$, that is, is made permanent since it might be the subpath of an efficient s - t -path p^* . In this case p is needed when the algorithm terminates to be able to reconstruct p^* .

Search for a next queue path (Line 13 in Algorithm 1).

Assume that an s - v -path p^* is extracted from Q in Line 7 of Algorithm 1. Then, there is no s - v -path in Q . The procedure `nextQueuePath` (Algorithm 3) searches for a new queue path p_v^{new} for v among the s - v -paths in the lists $\mathcal{NQP}_{uv}$ for arcs $(u, v) \in \delta^-(v)$. After the search, there holds

$$p_v^{\text{new}} := \arg \min_{p \in \bigcup_{u \in \delta^-(v)} \mathcal{NQP}_{uv}} \text{lex} \{ p \mid \bar{c}(p) \neq \bar{c}(p^*) \text{ and } p \text{ fulfills Conditions 2 and 3 in Definition 11} \} \quad (2)$$

in case such a path exists. Since each $\mathcal{NQP}$ list is sorted in lex. increasing order w.r.t. $\bar{c}$ (see Theorem 4) we can look for p_v^{new} starting with the first element of each list and iterating over it until a candidate path is found or the list becomes empty (irrelevant

ALGORITHM 2. Propagate

Input : s - v -path p , priority queue Q , permanent paths $\mathcal{P}$, lists of explored paths $\mathcal{NQP}$, dominance bound β_t .

Output: Updated priority queue Q and a boolean flag telling if the propagation of p was successful along at least one outgoing arc of v .

```

1 Flag success  $\leftarrow$  False;;
2 for  $w \in \delta^+(v)$  do
3    $p_w^{new} \leftarrow p \circ (v, w)$ ;;
4   if not  $Q.contains(w)$  then
5     if  $\beta_t > \bar{c}(p_w^{new})$  and not  $\bar{c}_{dr}(\mathcal{P}_{st}) \leq_D \bar{c}_{dr}(p_w^{new})$  and not  $\bar{c}_{dr}(\mathcal{P}_{sw}) \leq_D \bar{c}_{dr}(p_w^{new})$  then
6       success  $\leftarrow$  True;;
7        $Q.insert(p_w^{new})$ ;;
8   else
9      $q \leftarrow Q.getPath(w)$  if  $\bar{c}(p_w^{new}) <_{lex} \bar{c}(q)$  then
10      if  $\beta_t > \bar{c}(p_w^{new})$  and not  $\bar{c}_{dr}(\mathcal{P}_{st}) \leq_D \bar{c}_{dr}(p_w^{new})$  and not  $\bar{c}_{dr}(\mathcal{P}_{sw}) \leq_D \bar{c}_{dr}(p_w^{new})$  then
11        success  $\leftarrow$  True;;
12         $Q.decreaseKey(w, p_w^{new})$ ;;
13        if not  $\bar{c}(p_w^{new}) \leq \bar{c}(q)$  then
14           $(x, w) \leftarrow$  last arc in path  $q$ ;;
15           $\mathcal{NQP}_{xw}.prepend(q)$ ; // Insert  $q$  at the beginning of  $\mathcal{NQP}_{xw}$ .
16      else
17        if not  $\bar{c}(q) \leq \bar{c}(p_w^{new})$  then
18          success  $\leftarrow$  True;;
19           $\mathcal{NQP}_{vw}.append(p_w^{new})$ ; // Insert  $p_w^{new}$  at the end of  $\mathcal{NQP}_{vw}$ .
20 return  $(Q, success)$ ;

```

ALGORITHM 3. nextQueuePath

Input : s - v -path p^* , lists of explored paths $\mathcal{NQP}$, arc set $\delta^-(v)$, permanent paths $\mathcal{P}$.

Output: New queue path for v , if one exists.

```

1  $p_v^{new} \leftarrow NULL$ ; // Assume  $\bar{c}(p_v^{new}) = [\infty]_{i=1}^d$ .
2 for  $u \in \delta^-(v)$  do
3   for  $p \in \mathcal{NQP}_{uv}$  do
4     if not  $\bar{c}(p) <_{lex} \bar{c}(p_v^{new})$  then break;
5     if  $\bar{c}(p^*) == \bar{c}(p)$  or  $\bar{c}_{dr}(\mathcal{P}_{st}) \leq_D \bar{c}_{dr}(p)$  or  $\bar{c}_{dr}(\mathcal{P}_{sv}) \leq_D \bar{c}_{dr}(p)$  then
6        $\mathcal{NQP}_{uv} \leftarrow$  remove  $p$  from  $\mathcal{NQP}_{uv}$ ;;
7       continue;
8   else
9      $p_v^{new} \leftarrow p$ ;;
10    break;
11 return  $p_v^{new}$ ;

```

paths are deleted during the iteration). Thus, nextQueuePath identifies at most one candidate s - v -path via each predecessor node of v . The path p_v^{new} returned by the search is a lex. smallest one among these candidates (Condition 1 in Definition 11), if there is any. Hence, if found, p_v^{new} is a queue path for v and is inserted into Q (Line 14 in Algorithm 1).

4.1 | Correctness

The following remark is important for the proofs in this section and explains why we can perform all dominance checks in propagate (Algorithm 2) and nextQueuePath (Algorithm 3) w.r.t. $\bar{c}$.

Remark. For any $v \in V$, the lex. order of two s - v -paths p and q is the same w.r.t. c and $\bar{c}$:

$$\bar{c}(p) <_{\text{lex}} \bar{c}(q) \Leftrightarrow c(p) + \pi(v) <_{\text{lex}} c(q) + \pi(v) \Leftrightarrow c(p) <_{\text{lex}} c(q).$$

The key to prove the correctness of the T-MDA is Theorem 5 that states that during the algorithm's execution, the $\bar{c}$ costs of the paths extracted in Line 7 of Algorithm 1 do not decrease. To prove it, we need to ensure that the novel handling of explored paths using $\mathcal{NQP}$ lists in the T-MDA maintains this property (Lemmas 3,4). After Theorem 5 is proven, the correctness of the T-MDA can be proven as it is done in the literature for other label setting MOSP algorithms [1,14,21].

Lemma 3. *As long as the $\mathcal{NQP}$ lists of explored paths are sorted in lex. non-decreasing order w.r.t. $\bar{c}$, the T-MDA extracts paths from Q in Line 7 in lex. non-decreasing order w.r.t. $\bar{c}$.*

Proof. Let p be the path extracted from Q in iteration $l \in \mathbb{N}$ of the T-MDA. Assume that in a prior iteration $k < l$, a path q is extracted from Q such that

$$\bar{c}(p) <_{\text{lex}} \bar{c}(q).$$

The T-MDA extracts a lex. minimal path from Q in every iteration. Thus, neither p nor any subpath of p can be in Q in iteration l since by Lemma 2 they are lex. smaller than p w.r.t. c and $\bar{c}$. However, in iteration k there must be a queue path p'_j for j in Q for at least one node $j \in p$. Assume j is the last such node along p . By Lemma 2 we know that the s - j -subpath p_j of p is lex. smaller than p and we must have $\bar{c}(p_j) \leq_{\text{lex}} \bar{c}(p'_j)$. Otherwise, we would have

$$\bar{c}(p'_j) <_{\text{lex}} \bar{c}(p_j) <_{\text{lex}} \bar{c}(p) <_{\text{lex}} \bar{c}(q).$$

Since p'_j and q are in Q in iteration k , this would contradict q 's extraction.

Recall that p_j is an explored s - j -path but it is not the queue path for j in iteration k . We discuss the different cases that could lead to p'_j being in Q instead of p_j and conclude that none of them can actually happen.

- Case 1: The path p_j is replaced by p'_j in Q in a decreaseKey operation in Line 12 of Algorithm 2. This cannot happen since p_j is lex. smaller than p'_j and decreaseKey operations only happen if the lex. check in Line 9 of Algorithm 2 succeeds.
- Case 2: During the iteration wherein p_j is explored for the first time, p'_j is in Q . In this situation, a decreaseKey operation is triggered in Line 12 of Algorithm 2 to replace p'_j with p_j in Q . After this happens, p_j only leaves Q if it is either extracted or if a newly explored s - j -path is lex. smaller (cf. Case 1). Thus, p'_j cannot be extracted from Q prior to p_j .
- Case 3: In some iteration prior to iteration k , an s - j -path p''_j is extracted from Q and a new queue path for j is searched in Algorithm 3. Let $(i, j) \in A$ be the last arc of p_j and $(i', j) \in A$ the last arc of p'_j . We can assume that p_j and p'_j are in the lists $\mathcal{NQP}_{ij}$ and $\mathcal{NQP}_{i'j}$ respectively and are considered as queue path candidates during the search. Since p'_j is in Q during iteration k and p_j is extracted from Q in an iteration before the l th, but after the k th (if not, p would not be extracted in iteration l), both s - j -paths are neither dominated at $\mathcal{P}_{sj}$ w.r.t. c nor at $\mathcal{P}_{st}$ w.r.t. $\bar{c}$ (Line 6 of Algorithm 3). Otherwise, they would be permanently discarded in Line 6 of Algorithm 3. Recall that the new queue path p_v^{new} for node j computed by the search is lex. minimal among the queue path candidates in the $\mathcal{NQP}$ lists. Given that the lex. ordering of the $\mathcal{NQP}$ lists is assumed in the statement from the lemma, it cannot happen that $p_v^{\text{new}} = p'_j$ at the end of the search if p_j is in $\mathcal{NQP}_{ij}$ as assumed.

We conclude that the existence of the explored path p_j avoids p'_j or any other s - j -path that is lex. greater than p_j from being in Q in iteration k . However, there must be an s - j -path in Q during this iteration. This implies that q would not be extracted from Q since the s - j -path in Q is lex. smaller. ■

Lemma 4. *If paths are extracted from Q in Line 7 of the T-MDA in lex non-decreasing order w.r.t. $\bar{c}$, the $\mathcal{NQP}$ lists of explored paths remain sorted in lex non-decreasing order w.r.t. $\bar{c}$.*

Proof. Clearly, as long as the $\mathcal{NQP}$ lists contain at most one element, they are sorted. Let the current iteration be the first one wherein a second path is added to an $\mathcal{NQP}$ list, say $\mathcal{NQP}_{vw}$, $(v, w) \in A$. There are two ways in which this can occur.

- Case 1: A new s - w -path q is prepended to $\mathcal{NQP}_{vw}$ in Line 15 of Algorithm 2. The path q was in Q until the current iteration and is now replaced by a newly explored path p_w^{new} s.t. $\bar{c}(p_w^{\text{new}}) <_{\text{lex}} \bar{c}(q)$ (Line 9 of Algorithm 2). Let q' be

the s - w -path that is already in $\mathcal{NQP}_{vw}$. As a consequence of the third queue path condition, q' is not lex. smaller than q . Thus, adding q at the beginning of $\mathcal{NQP}_{vw}$ does not alter its ordering.

Case 2: A newly explored s - w -path p_w^{new} is appended to $\mathcal{NQP}_{vw}$ in Line 19 of Algorithm 2. By construction, (v, w) is the last arc of p_w^{new} and of the path q' , the s - w -path via v that is already in $\mathcal{NQP}_{vw}$. Note that the s - v -subpath p of p_w^{new} is extracted from Q in the current iteration. Since the $\mathcal{NQP}$ lists were sorted until the current iteration, Lemma 3 guarantees that paths are extracted from Q in lex. non decreasing order w.r.t. $\bar{c}$. As a consequence, the s - v -subpath q'_v of q' must have been extracted prior to p and $\bar{c}(q'_v) \leq_{lex} \bar{c}(p)$. This implies $c(q'_v) \leq_{lex} c(p)$ and since both s - v -subpaths are expanded along the same arc (v, w) to obtain q' and p_w^{new} , we have $c(q') \leq_{lex} c(p_w^{new})$. Thus, adding p_w^{new} after q' to the list $\mathcal{NQP}_{vw}$ does not alter the ordering of the list.

An induction over the number of elements in the $\mathcal{NQP}$ lists proves the lemma. ■

The last two lemmas show that the ordering of the $\mathcal{NQP}$ lists and of the priority queue Q influence each other. We can combine both statements to obtain the following key lemma for the correctness of the T-MDA.

Theorem 5. *Let p be a path extracted in Line 7 of Algorithm 1. Then, for all paths q that were extracted from Q prior to p we have*

$$\bar{c}(q) \leq_{lex} \bar{c}(p).$$

Proof. Lemma 3 trivially holds until the first iteration wherein an $\mathcal{NQP}$ list gets a second path. Then, Lemma 4 guarantees that this path is inserted correctly into its $\mathcal{NQP}$ list, which then satisfies the condition of Lemma 3 to guarantee that in the upcoming iterations, paths are still extracted in lex. non decreasing order w.r.t. $\bar{c}$. Repeating these arguments until the last iteration of the T-MDA proves the lemma. ■

As a consequence of the last lemma, the permanent s - v -paths in $\mathcal{P}_{sv}$ are also sorted in lex. increasing order w.r.t. $\bar{c}$ and w.r.t. c . We need the following lemma to finally prove the correctness of the T-MDA.

Lemma 6. *Consider a node $v \in V$. Any s - v -path p extracted from Q in Line 7 of Algorithm 1 is an efficient s - v -path.*

Proof. Given Theorem 5, the statement can be proven as in [14] [Lemma 1]. ■

Theorem 7. *At the end of Algorithm 1, the set $\mathcal{P}_{st}$ is a minimal complete set of efficient s - t -paths.*

The proof of the last theorem is very similar to that of Theorem 1 in [14]. Given Lemma 6, we only have to show that for every non-dominated cost vector exactly one s - t -path is found in the T-MDA. The iterative building of $\mathcal{P}_{st}$ and the usage of the reflexive $\leq_D$ -relation in the dominance checks ensure that at most one s - t -path is found for every non-dominated cost vector. The lex. ordered extraction of paths from Q (Theorem 5) and the admissibility of the heuristic π are used to prove that for every non-dominated cost vector an efficient s - t -path is found.

4.2 | Avoiding the repetition of dominance checks

The T-MDA as introduced in this paper might check the dominance of a path p in Line 5 or Line 10 of Algorithm 2 and multiple times in Line 5 of Algorithm 3. As explained in [14] [section 3.2] these dominance checks can be repeated up to $N_{\max}$ times for p , where $N_{\max} := \max_{v \in V} |\mathcal{P}_{sv}|$.

Assume that an s - v -path p is considered $k \leq N_{\max}$ times in Algorithm 3 and suppose the current iteration checks the dominance of p for the l th time for some $l \leq k$. Then, all vectors contained in the fronts $\bar{c}_{dr}(\mathcal{P}_{st})$ and $\bar{c}_{dr}(\mathcal{P}_{sv})$ the $(l-1)$ th time p was processed still do not dominate p . Otherwise, p would have been discarded in Line 6 of Algorithm 3. Hence, to decide whether p is dominated by any permanent s - v - or s - t -path it suffices to consider the permanent paths at the corresponding fronts that were added between iteration $(l-1)$ and l , if there are any. How this can be efficiently implemented can be seen in our open source implementation of the T-MDA. Overall, checking if p is dominated by an efficient s - v -path requires $\mathcal{O}(dN_{\max})$ comparisons regardless of how often p is considered in different calls to Algorithm 3.

4.3 | Complexity of the searches for new queue paths

Let n and m be the number of nodes and arcs in G , respectively. Additionally, we set N to be the total number of extracted paths during the T-MDA and, again, $N_{\max} := \max_{v \in V} |\mathcal{P}_{sv}|$ to be the maximum cardinality of a list of permanent paths at the end of the algorithm.

Up until now, we could ignore the mandatory question of how paths should be stored efficiently. If p is an s - v -path and (u, v) its last arc, p can be uniquely encoded as a tuple that stores p 's end node v and a reference to the tuple encoding the s - u -subpath of p . The possibility to store paths using these tuples, often called *labels*, allows us to assume that a path's memory consumption in the T-MDA algorithm is $\mathcal{O}(1)$. Given a path's label, the path itself can be reconstructed recursively after the algorithm terminates. Since the T-MDA algorithm only outputs simple paths, this can be achieved in $\mathcal{O}(n)$ time per path.

Using the $\mathcal{NQP}$ lists has no impact on the running time bounds for the original MDA derived in [14], that is, they remain valid for the T-MDA. In the original MDA these lists do not exist and permanent paths are re-expanded multiple times in a routine similar to Algorithm 3 called *nextCandidatePath*. The required sums to rebuild the cost vectors of re-expanded paths do not impact the running time bound of the MDA. The paths that are rebuilt in the *nextCandidatePath* procedure of the MDA are the ones stored in the $\mathcal{NQP}$ lists of 1. Regardless of whether the paths are rebuilt or they are picked from an $\mathcal{NQP}$ list, the processing of the paths to enter Q or to discard them is the same in the original MDA and in the new T-MDA.

In practice, the re-expansion of paths has an impact on the algorithms' running times, which motivates the usage of $\mathcal{NQP}$ lists to avoid having to repeat the expansion of permanent paths. For every arc $(v, w) \in A$ the T-MDA maintains an $\mathcal{NQP}_{vw}$ list that stores the expansions of permanent s - v -paths along (v, w) that are still not dominated or have not been checked for dominance yet. In the worst case, these are the expansions of all paths in $\mathcal{P}_{sv}$. Thus, the $\mathcal{NQP}$ lists require an additional storage space of $\mathcal{O}(mN_{\max})$ since we assume that paths are stored as labels. Moreover, the $\mathcal{NQP}$ lists need to remain sorted, but as shown in Lemma 4, their ordering can be maintained without extra computational effort by always prepending or appending new paths to the lists.

In contrast to the One-To-All scenario, it is not possible to design an algorithm for the One-to-One MOSP problem that runs in polynomial time w.r.t. the input size and the output size of any given instance [4]. Still, it is worth noting that the running time bounds derived in [21] and [14] for the MDA are still valid for the new T-MDA despite the increased memory consumption caused by the $\mathcal{NQP}$ lists.

5 | NAMOA^{*}_{DR}-LAZY ALGORITHM

The NAMOA^{*}_{dr} algorithm was introduced in [18] and, to the best of our knowledge, it constitutes the state-of-the-art for A^{*} based algorithms for MOSP problems. It equips the original NAMOA^{*} algorithm from [17] with the dimensionality reduction technique discussed in Section 2. Similarly to the T-MDA, the algorithm uses a priority queue Q of paths. In the NAMOA^{*}_{dr} algorithm, the coexistence of multiple s - v -paths in Q is allowed as long as they do not dominate each other. To ensure this invariant, the NAMOA^{*}_{dr} algorithm uses merge operations (cf. Definition 6) after exploring new paths and checking that they are not dominated by any permanent path. The computational results in [14] substantiate that in One-to-One MOSP label setting MOSP algorithms, merge operations during the expansions of paths should be avoided to obtain competitive running times.

Inspired by the label setting algorithm for Biobjective Shortest Path problems presented in [1] that avoids merge operations, we introduce the NAMOA^{*}_{dr}-lazy algorithm. The pseudocode is given in Algorithm 4. Explored paths are handled in a lazy way. This means that every path that is not dominated by existing permanent paths (Line 14) when it is analyzed for the first time is inserted into the algorithm's priority queue (Line 17). Hence, the queue can contain exponentially many paths simultaneously and for any $v \in V$, s - v -paths in the queue can dominate each other. As a consequence and in contrast to the invariant regarding the efficiency of extracted paths in the T-MDA, paths that are extracted from the queue (Line 6) are not guaranteed to be efficient. To avoid the storage and expansion of such paths, the algorithm needs to check if they are dominated right after their extraction. This is achieved as a side-effect of the call to the merge procedure to update the dimensionality reduced fronts in Line 8. Assume the s - v -path p is extracted from Q . A merge operation (cf. Definition 6) possibly updates the front $\bar{c}_{dr}(\mathcal{P}_{sv})$ by inserting $\bar{c}_{dr}(p)$ and deleting elements dominated by $\bar{c}_{dr}(p)$. However, if $\bar{c}_{dr}(p)$ is not added to the front, p is dominated (cf. Theorem 1).

6 | EXPERIMENTS

We compare the performance of the T-MDA (Algorithm 1) and the NAMOA^{*}_{dr}-lazy algorithm (Algorithm 4) on 3-dimensional One-to-One MOSP instances. We decided not to benchmark the original NAMOA^{*}_{dr} algorithm from [18]. In Figure 1, we show the results of the direct comparison of the new NAMOA^{*}_{dr}-lazy algorithm and our implementation of the original NAMOA^{*}_{dr} algorithm on the road network of New York. Already on this network (the smallest considered road network) it becomes apparent that the NAMOA^{*}_{dr}-lazy algorithm performs better consistently. The reason is that the original NAMOA^{*}_{dr} algorithm conducts merge operations after every successful expansion of a path along one of its outgoing arcs. On the instances showed in Figure 1 the NAMOA^{*}_{dr}-lazy algorithm is 1.96 times faster on average. If we consider only the instances with an output set that contains more than 1000 paths, the new algorithm is already 2.51 times faster on average.

ALGORITHM 4. NAMOA_{dr}^{*}-lazy

Input : MOSP instance $\mathcal{I} = (G, s, t, c)$, heuristic π for $\mathcal{I}$, dominance bound β_i for $\mathcal{I}$.

Output: Minimal complete set $\mathcal{P}_{st}$ of efficient s - t -paths.

```

1 Priority queue of paths  $Q \leftarrow \emptyset$ ; // Sorted according to  $\bar{c}$ .
2  $\forall v \in V$  – efficient  $s$ - $v$ -paths:  $\mathcal{P}_{sv} \leftarrow \emptyset$ ;
3  $p_{\text{init}} \leftarrow ()$ ;
4  $Q \leftarrow Q.\text{insert}(p_{\text{init}})$ ;
5 while  $Q \neq \emptyset$  do
6    $p \leftarrow Q.\text{extractMin}()$ ;
7    $v \leftarrow$  last node of path  $p$ .; // If  $p = p_{\text{init}}$ ,  $v \leftarrow s$ .
8    $\bar{c}_{dr}(\mathcal{P}_{sv}) \leftarrow \text{merge}(\bar{c}_{dr}(\mathcal{P}_{sv}), \bar{c}_{dr}(p))$ ;
9   if  $\bar{c}_{dr}(p) \notin \bar{c}_{dr}(\mathcal{P}_{sv})$  then continue;
10  Flag success  $\leftarrow$  False;
11  if  $v \neq t$  then
12    for  $w \in \delta^+(v)$  do
13       $p_w^{\text{new}} \leftarrow p \circ (v, w)$ ;
14      if  $\beta_t \leq \bar{c}(p_w^{\text{new}})$  or  $c_{dr}(\mathcal{P}_{st}) \leq_D \bar{c}_{dr}(p_w^{\text{new}})$  or  $\bar{c}_{dr}(\mathcal{P}_{sw}) \leq_D \bar{c}_{dr}(p_w^{\text{new}})$  then
15        continue
16      else
17         $Q.\text{insert}(p_w^{\text{new}})$ ;
18        success  $\leftarrow$  True;
19  if  $v == t$  or success == True then  $\mathcal{P}_{sv}.\text{append}(p)$ ;
20 return  $\mathcal{P}_{st}$ ;

```

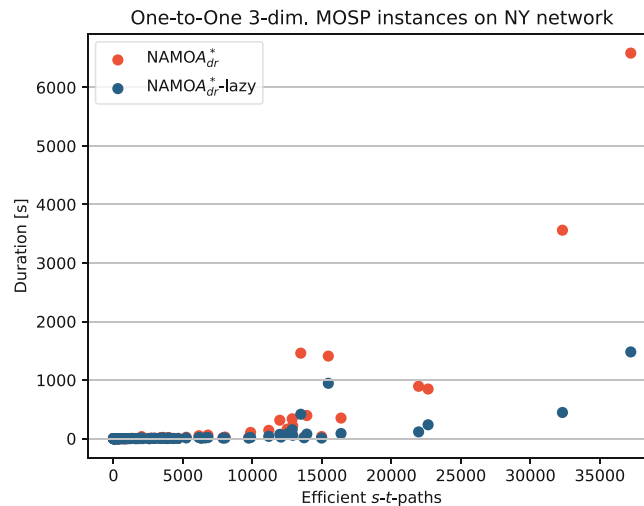


FIGURE 1 Comparison on NY network. Linear scales used to emphasize the superiority of the NAMOA_{dr}^{*}-lazy algorithm.

In our experiments, the algorithms get the same heuristic as part of their input and hence they require the same preprocessing. Moreover, the NAMOA_{dr}^{*}-lazy algorithm and the T-MDA use a binary heap as their priority queue. We always benchmark the time needed to find a minimal complete set only, that is, no preprocessing times are benchmarked in our experiments. Both algorithms are implemented in C++ and are publicly available in [12]. The repository also contains our implementation of the MDA algorithm [14] and of the NAMOA_{dr}^{*} algorithm [18]. We used the gcc 7.5.0 compiler with the optimization flag O3 to build the binaries. The experiments were performed on a computer with an Intel Xeon Gold 5122 3.60GHz processor and 96 GB memory. The time limit for every instance was set to 5 h.

6.1 | Instance description

NetMaker Instances $d = 3$

Netmaker graphs are synthetic graphs with 5000 to 30 000 nodes and 29 591 to 688 398 arcs. These graphs have been used in multiple publications [14,19,20,22]. In every such graph, all nodes are connected via a Hamiltonian cycle to ensure connectivity. Then, arcs between the nodes along the cycle are added randomly. The arc costs are three dimensional and each cost component lies between 1 and 1000. For any arc there is a cost component between 1 and 333, a cost component between 334 and 666, and a cost component between 667 and 1000. Which cost component lies in which interval and the actual value of the cost component was randomly set when the graphs were first created. We group the NetMaker graphs into groups called *netM-n*. Each group is defined by the number $n10^3$ of nodes of the graphs in it (i.e., the netM-5 group contains graphs with 5000 nodes) and contains 10 to 12 graphs with varying number of arcs. We use the same s - t -pairs used in [14]: for each graph, 20 randomly generated s - t -pairs are considered. Thus, in every netM- n group 200–240 instances are considered.

Road Networks $d = 3$

We use the well known road networks of (parts of) the United States available from the DIMACS Implementation Challenge on Shortest Paths [5]. The original instances are two-dimensional: each arc in the network represents a street and the costs of the arc are the street's length and traversal time. We add a third cost component that is equal to one on every arc. In Table 1, we list the sizes of the used networks. For the experiments we use the same s - t -pairs (100 per road network) that were used in [1].

6.2 | Results

The results of the experiments are summarized in Tables 2 and 3. Throughout the section we use scatter plots to visualize the results. We plot the instances' running time depending on the size of the computed minimum complete set of efficient s - t -paths. Both axes use a logarithmic scale. The raw datasets containing the results and the scripts used to evaluate them can be found in [12].

Remark (Speedups). The reported speedups in the rightmost columns of Tables 2 and 3 are calculated by building the speedup for every instance in the considered group of instances and then calculating the arithmetic mean of these results. Moreover, to calculate the speedups we divide the running time of the NAMOA_{dr}^{*}-lazy algorithm by the running time of the T-MDA. Thus, speedups that are greater than one correspond to groups of instances in which the T-MDA is faster and are reported in bold font.

6.2.1 | General observations

For every netM- n group of graphs and for every road network, we partition the solved instances into intervals depending on the cardinality $|\mathcal{P}_{st}|$ of the output sets $\mathcal{P}_{st}$. This way, we obtain a better insight into the asymptotic behavior of the algorithms' running times. Regardless of the type of instance (netMaker or road) we observe that the NAMOA_{dr}^{*}-lazy algorithm is faster than the T-MDA on instances whose output is small. Such instances require few iterations to be solved. Hence, even if the NAMOA_{dr}^{*}-lazy algorithm inserts more paths than the T-MDA algorithm into the heap, the heap can be handled efficiently. In this situation, maintaining the $\mathcal{NQP}$ lists in the T-MDA turns out to be less efficient than inserting every relevant explored path into the heap. As the size of the output sets $\mathcal{P}_{st}$ increases, instances are solved faster by the T-MDA. Here, keeping the

TABLE 1 Size of the used road networks.

Road network	Nodes	Arcs
NY	264 346	733 846
BAY	321 270	800 172
COL	435 666	1 057 066
FLA	1 070 376	2 712 798
NE	1 524 453	3 897 636
LKS	2 758 119	6 885 658
E	3 598 623	8 778 114
W	6 262 104	15 248 146
CTR	14 081 816	34 292 496

Note: In every network, we consider 100 s - t -pairs from [1].

TABLE 2 T-MDA versus NAMOA_{dr}^{*}-lazy algorithm on NetMaker graphs.

	$ \mathcal{P}_{st} $ in range	Instances	$ \mathcal{P}_{st} $ avg.	T-MDA		NAMOA _{dr} [*] lazy		Speedup
				Inserted avg.	Time avg.	Inserted avg.	Time avg.	
netM-5	(0, 100]	11	41	3260	<0.01	4427	<0.01	0.30
	(100, 1000]	128	500	177 649	0.39	220 350	0.47	0.95
	(1000, 5000]	95	1830	867 209	3.17	1 049 310	4.53	1.29
	(5000, 10 000]	6	5988	1 674 750	7.91	2 010 315	12.61	1.52
netM-10	(0, 100]	4	49	3365	<0.01	4731	<0.01	0.34
	(100, 1000]	106	508	198 375	0.33	247 446	0.35	0.91
	(1000, 5000]	102	2221	1 791 187	8.77	2 149 593	13.82	1.33
	(5000, 10 000]	8	5541	4 980 142	29.65	5 737 303	50.05	1.64
netM-15	(0, 100]	5	91	16 443	0.01	24 157	0.01	0.70
	(100, 1000]	98	541	300 291	0.62	381 555	0.74	0.93
	(1000, 5000]	123	2241	2 533 303	14.66	3 082 628	24.08	1.39
	(5000, 10 000]	14	6081	7 490 833	58.06	8 628 046	103.55	1.71
netM-20	(0, 100]	3	78	13 670	0.01	17 842	0.01	0.40
	(100, 1000]	91	531	379 213	0.92	477 129	1.17	0.94
	(1000, 5000]	131	2367	3 006 318	17.39	3 670 609	28.34	1.40
	(5000, 10 000]	15	6337	8 938 319	72.56	10 692 976	139.45	1.77
netM-25	(0, 100]	4	34	5350	0.01	7248	<0.01	0.21
	(100, 1000]	89	576	504 913	1.41	637 491	1.84	0.93
	(1000, 5000]	93	2499	5 110 110	36.65	6 241 874	65.02	1.50
	(5000, 10 000]	14	5878	10 427 418	96.20	12 554 838	179.89	1.87
netM-30	(0, 100]	2	42	852	0.01	1244	<0.01	0.08
	(100, 1000]	90	554	414 782	0.86	522 768	0.94	0.93
	(1000, 5000]	125	2409	4 247 590	27.86	5 164 629	46.59	1.42
	(5000, 10 000]	22	6334	12 850 042	112.95	15 303 492	212.96	1.82
	(10 000, 1 000 000]	1	11 566	1 347 096	5.89	1 700 574	9.40	1.60

Note: Instances are separated depending on the cardinality of the computed minimal complete sets $\mathcal{P}_{st}$ of efficient paths. The cardinality $|\mathcal{P}_{st}|$ of these sets and the numbers in the Inserted, Time, and Speedup columns are arithmetic means. In the $|\mathcal{P}_{st}|$ and Inserted columns we rounded the averages to become integers.

size of the heap bounded by holding back explored paths in the $\mathcal{NQP}$ lists of the T-MDA to possibly discard them without ever entering the queue turns out to be a consistent advantage. This effect can also be clearly observed in the scatter plots. On NetMaker instances with an output set of more than 1000 paths, the T-MDA is 1.29 to 1.87 times faster than the NAMOA_{dr}^{*}-lazy algorithm. Regarding the instances on road networks, the T-MDA is 1.02 to 1.33 times faster on instances with an output set containing more than 5000 paths. In the next two sections, we discuss the results in more detail.

6.2.2 | NetMaker graphs

The results corresponding to the NetMaker instances are summarized in Table 2. The scatter plots showing the comparison of the running times of both algorithms on every instance are shown in Figures 2–7. All instances were solved by both algorithms in less than 5 h.

Remark. Note that on the lower left part of the NetMaker plots some points representing NAMOA_{dr}^{*}-lazy solutions are missing. This is because the output from our source code parses running times of less than 0.001 s as zeros. The NAMOA_{dr}^{*}-lazy algorithm was of course able to solve this very small instances.

In every netM- n group we observe that the the T-MDA performs better when the number of computed efficient paths grows. On all groups of instances that can be solved in less than 2s the NAMOA_{dr}^{*}-lazy algorithm wins the comparison. The greatest speedup in favor of the NAMOA_{dr}^{*}-lazy algorithm is achieved on two instances in the netM-30 group whose output contains 42 paths on average. These instances are solved by both algorithms in not more than 0.01 s but the NAMOA_{dr}^{*}-lazy algorithm is more than an order of magnitude faster. On groups of instances that are solved in more than 2 s on average, the T-MDA outperforms the NAMOA_{dr}^{*}-lazy algorithm consistently. It reaches an average speedup factor of more than 1.8 on the instances with the largest number of efficient solutions. In Figures 2–7 it becomes apparent that the speedup in favor of the T-MDA grows with the output size on the netMaker instances.

TABLE 3 T-MDA versus NAMOA_{dr}^{*}-lazy algorithm on road networks.

	$ \mathcal{P}_{st} $		T-MDA			NAMOA _{dr} [*] lazy			
	Range	Avg.	Solved	Inserted Avg.	Time Avg.	Solved	Inserted Avg.	Time Avg.	Speedup
NY	(0, 100]	34	15/15	3321	0.01	15/15	3546	<0.01	0.08
	(100, 1000]	405	31/31	110 712	0.07	31/31	120 616	0.05	0.57
	(1000, 5000]	2909	29/29	2 390 059	2.74	29/29	2 568 089	2.99	1.02
	(5000, 10 000]	7402	9/9	7 314 040	10.44	9/9	7 782 472	12.63	1.19
	(10 000, 1 000 000]	17 220	16/16	48 791 057	191.14	16/16	50 904 578	265.49	1.32
BAY	(0, 100]	33	9/9	5082	0.01	9/9	5313	<0.01	0.09
	(100, 1000]	371	28/28	201 168	0.14	28/28	209 201	0.11	0.61
	(1000, 5000]	2448	26/26	2 577 514	4.03	26/26	2 670 647	4.15	0.94
	(5000, 10 000]	7144	11/11	18 748 731	74.04	11/11	19 194 221	93.17	1.14
	(10 000, 1 000 000]	28 670	26/26	193 531 796	2043.29	26/26	196 455 631	2804.31	1.28
COL	(0, 100]	39	12/12	7412	0.01	12/12	7714	<0.01	0.10
	(100, 1000]	508	16/16	241 376	0.15	16/16	256 349	0.11	0.66
	(1000, 5000]	2004	21/21	1 955 302	2.38	21/21	2 009 058	2.38	0.94
	(5000, 10 000]	7574	10/10	10 112 780	23.32	10/10	10 386 419	27.75	1.12
	(10 000, 1 000 000]	48 448	37/37	246 331 650	2378.92	37/37	249 890 196	3169.22	1.28
FLA	(0, 100]	38	6/6	2692	0.03	6/6	2910	<0.01	0.02
	(100, 1000]	467	8/8	179 462	0.12	8/8	208 016	0.09	0.51
	(1000, 5000]	3679	10/10	14 376 178	31.84	10/10	14 775 863	38.67	1.10
	(5000, 10 000]	6486	9/9	11 118 027	25.09	9/9	11 716 810	29.08	1.12
	(10 000, 1 000 000]	61 417	43/43	345 788 113	3822.60	39/43	271 936 388	5135.19	1.33
LKS	(0, 100]	2	1/1	97	0.07	1/1	97	<0.01	<0.01
	(100, 1000]	571	2/2	381 876	0.37	2/2	393 698	0.24	0.51
	(1000, 5000]	2276	5/5	1 044 454	1.06	5/5	1 122 525	0.96	0.87
	(5000, 10 000]	7080	4/4	21 100 218	108.51	4/4	21 587 255	133.98	1.16
	(10 000, 1 000 000]	64 768	17/17	297 427 996	3054.73	17/17	303 262 129	4100.09	1.29
NE	(0, 100]	34	1/1	2220	0.04	1/1	2262	<0.01	0.01
	(100, 1000]	412	7/7	161 246	0.15	7/7	167 479	0.08	0.38
	(1000, 5000]	3069	13/13	4 203 939	7.12	13/13	4 412 026	7.48	0.96
	(5000, 10 000]	6507	9/9	9 929 695	20.02	9/9	10 486 800	23.36	1.10
	(10 000, 1 000 000]	63 269	47/47	443 520 972	3549.03	44/47	369 787 382	4508.29	1.30
E	(100, 1000]	450	5/5	222 937	0.29	5/5	228 413	0.16	0.38
	(1000, 5000]	2828	7/7	5 102 816	10.67	7/7	5 254 239	11.53	0.88
	(5000, 10 000]	7032	4/4	9 763 375	16.78	4/4	10 271 645	19.85	1.12
	(10 000, 1 000 000]	63 621	24/24	386 436 905	3326.37	23/24	35 7574 955	4352.75	1.30
W	(1000, 5000]	2962	3/3	1 747 477	1.64	3/3	1 800 400	1.43	0.80
	(5000, 10 000]	7207	1/1	17 269 805	23.89	1/1	17 520 309	24.42	1.02
	(10 000, 1 000 000]	82 084	17/17	586 806 511	6879.18	14/17	493 275 262	8803.06	1.28
CTR	(1000, 5000]	4146	1/1	14 486 309	45.60	1/1	14 557 088	51.17	1.12
	(5000, 10 000]	8388	2/2	24 257 334	88.16	2/2	25 307 068	106.33	1.15
	(10 000, 1 000 000]	97 563	8/8	576 133 946	6559.87	6/8	323 618 647	7229.40	1.21

Note: Instances are separated depending on the cardinality of the computed minimal complete sets $\mathcal{P}_{st}$ of efficient paths. The cardinality $|\mathcal{P}_{st}|$ of these sets and the numbers in the Inserted, Time, and Speedup columns are arithmetic means. In the $|\mathcal{P}_{st}|$ and Inserted columns we rounded the averages to become integers.

There is only one instance in the netM-30 group whose output contains more than 10 000 s - t -paths. Interestingly, this instance is solved extremely fast by both algorithms. This instance can be considered an outlier.

6.3 | Road networks

The results corresponding to the instances on road networks are summarized in Table 3. The scatter plots showing the comparison of the running times of both algorithms on every instance are shown in Figures 8–16. The considered networks are larger than in the NetMaker experiments. However the cardinality of the solution sets grows slower w.r.t. the graphs' size. Intuitively

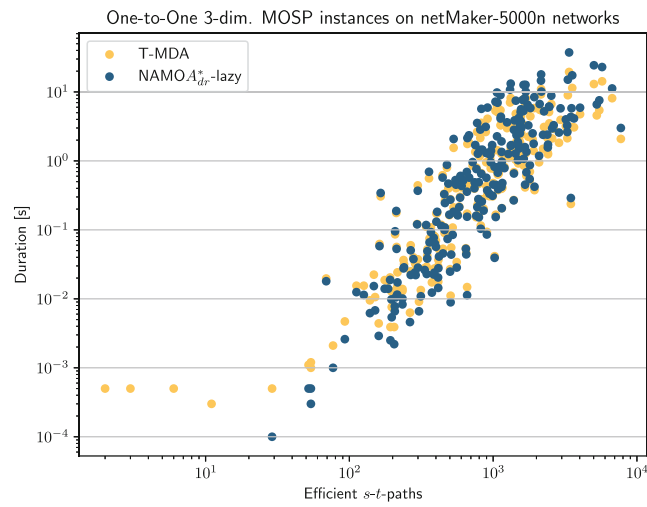


FIGURE 2 Comparison on netM-5 networks.

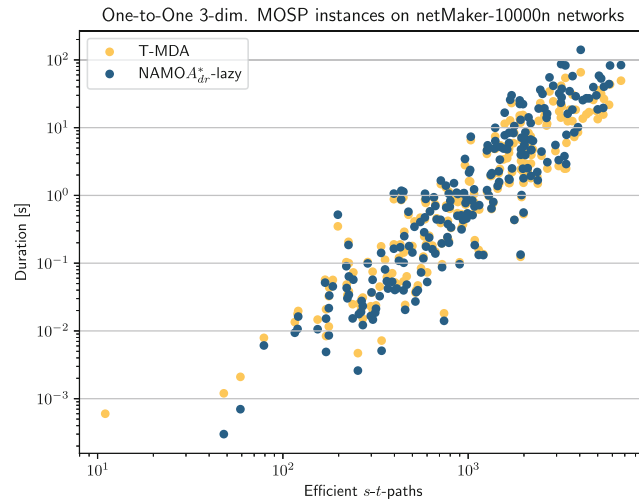


FIGURE 3 Comparison on netM-10 networks.

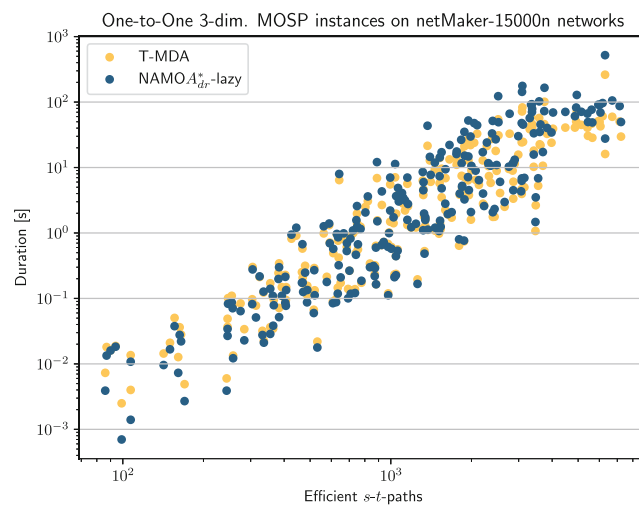


FIGURE 4 Comparison on netM-15 networks.

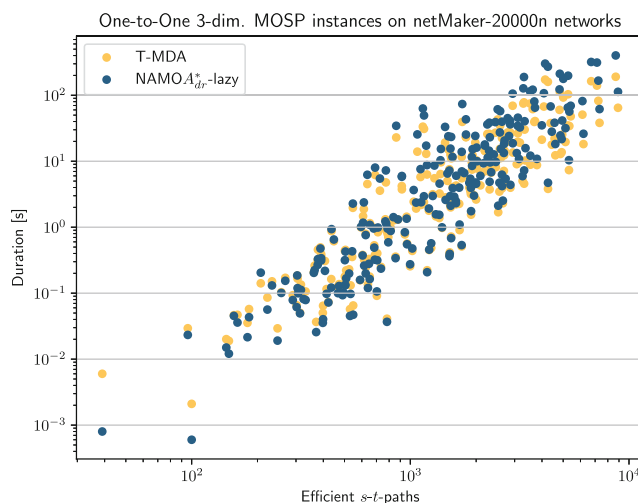


FIGURE 5 Comparison on netM-20 networks.

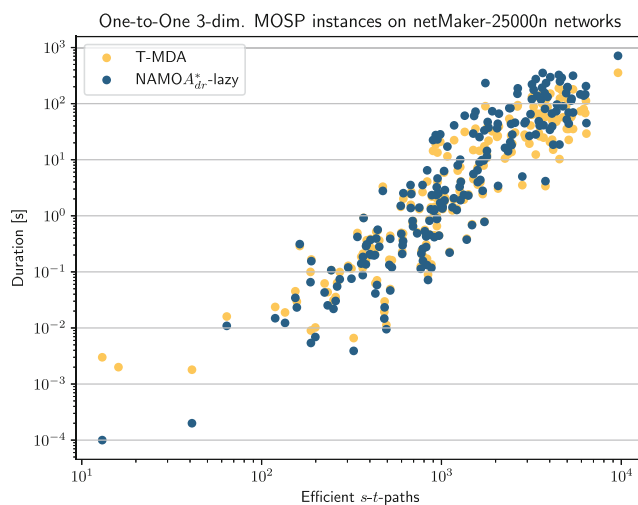


FIGURE 6 Comparison on netM-25 networks.

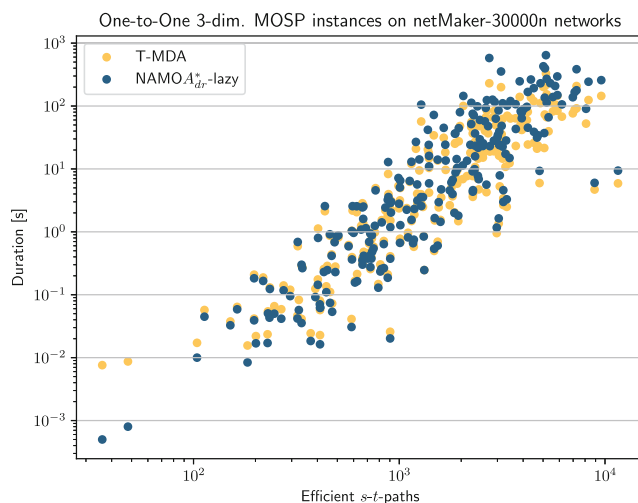


FIGURE 7 Comparison on netM-30 networks.

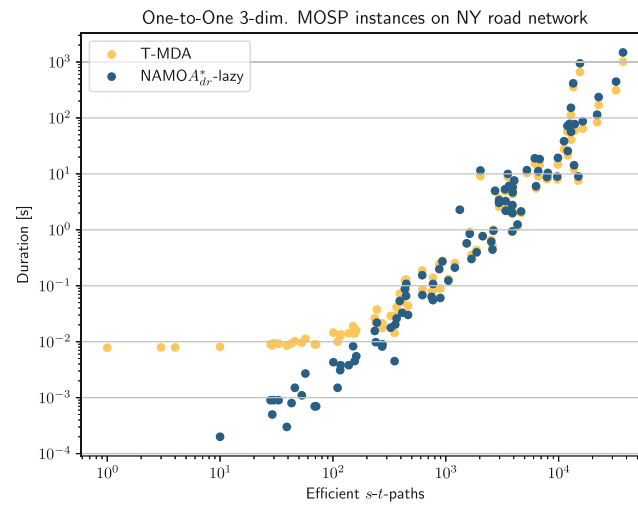


FIGURE 8 Comparison on NY network.

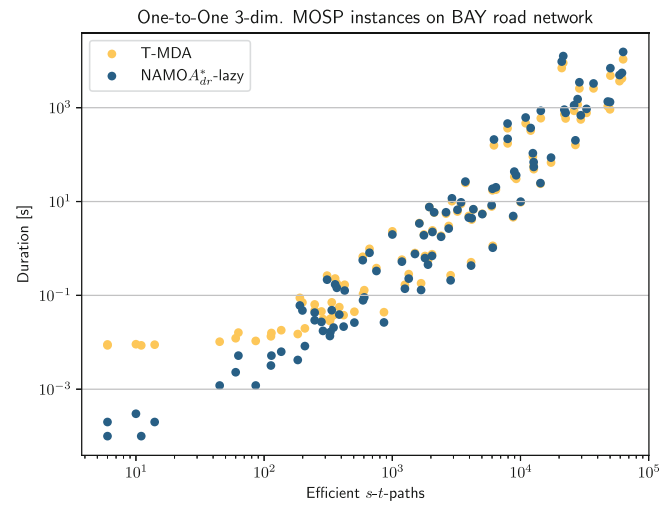


FIGURE 9 Comparison on BAY network.

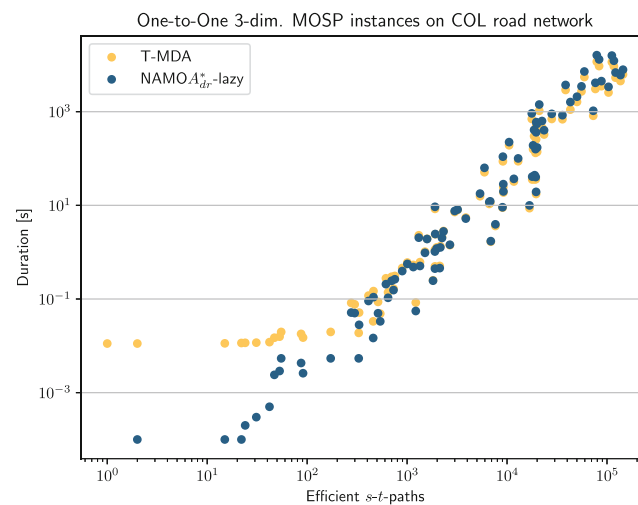


FIGURE 10 Comparison on COL network.

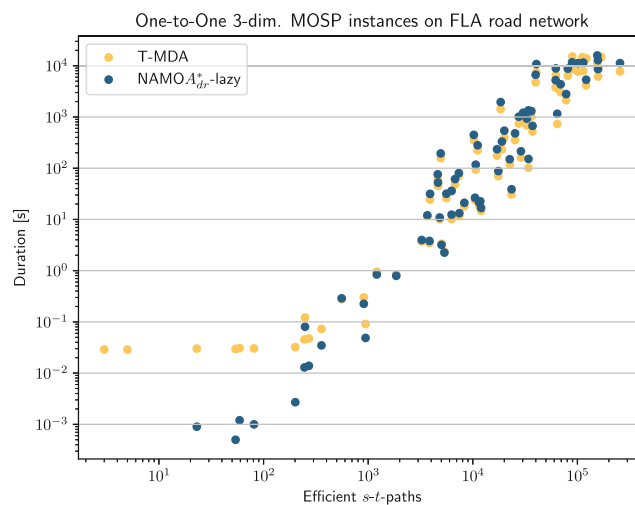


FIGURE 11 Comparison on FLA network.

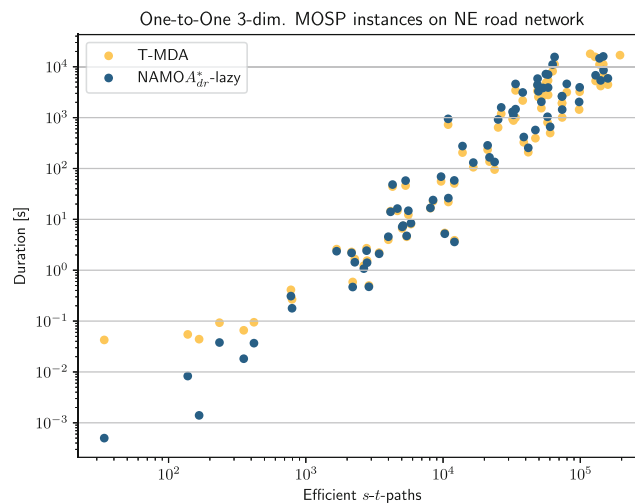


FIGURE 12 Comparison on NE network.

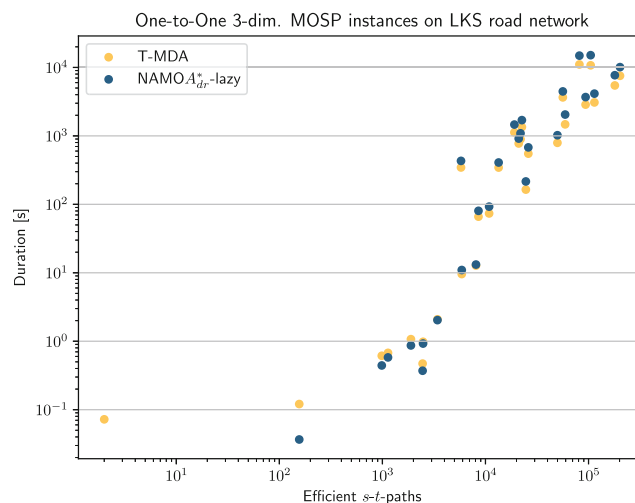


FIGURE 13 Comparison on LKS network.

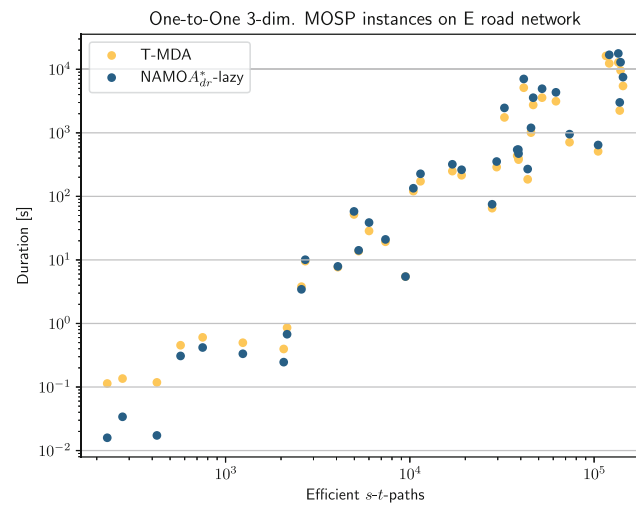


FIGURE 14 Comparison on E network.

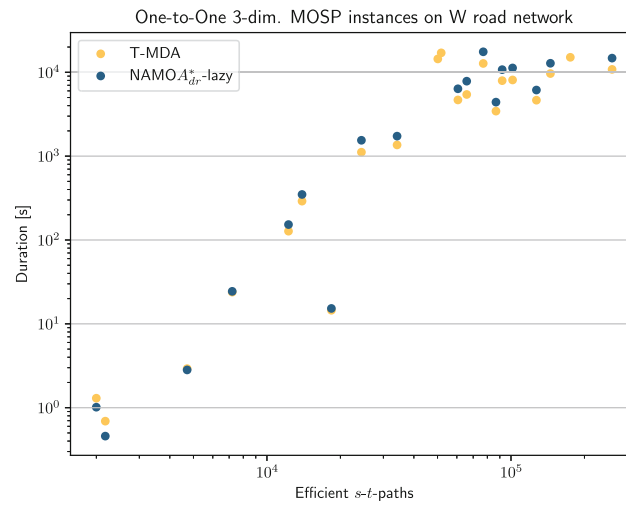


FIGURE 15 Comparison on W network.

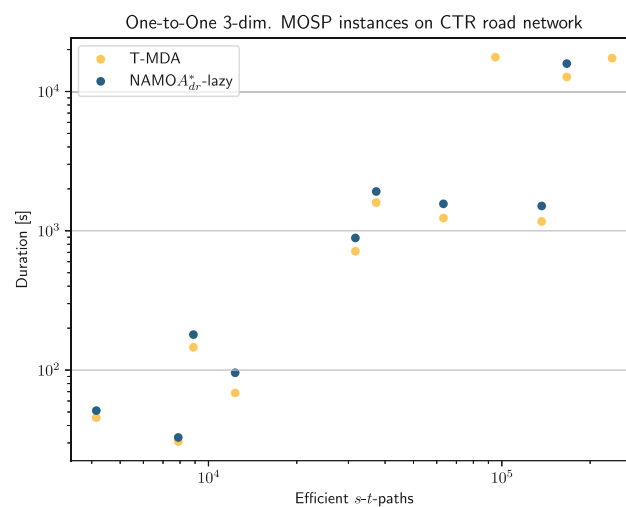


FIGURE 16 Comparison on CTR network.

the reason is that the arc costs encoding duration and length are correlated. Still, on the road networks with over two million arcs, not all instances are solved within the time limit.

Remark (Unsolved instances). Some instances were solved only by the T-MDA. We assume that NAMOA_{dr}^{*}-lazy algorithm needs 5 h to solve those instances. Instances that are not solved by any of the considered algorithms are ignored in Table 3. This handling of unsolved instances implies:

- 1 On some instance sets in which the NAMOA_{dr}^{*}-lazy algorithm solves less instances, artificial running times are set to 5 h. The actual NAMOA_{dr}^{*}-lazy running time on these instances would be greater.
- 2 The number of insertions made by both algorithms is not comparable on instance groups in which the NAMOA_{dr}^{*}-lazy algorithm solves less instances than the T-MDA. Whenever the NAMOA_{dr}^{*}-lazy algorithm does not solve an instance, we do not know the number of insertions that it needs to solve it. For example the last line in Table 3 suggests that the T-MDA does more insertions into the heap. This is only because the T-MDA solves more instances. Whenever the same set of instances is solved, the T-MDA does at most as many insertions as the NAMOA_{dr}^{*}-lazy algorithm.

In Table 3 we observe that, as noted in Section 6.2.1, the achieved speedups favor the T-MDA more when the instances become harder to solve, that is, on bigger graphs and on instances that require more iterations to be solved. However, the greatest speedups between 1.30 and 1.33 are smaller than the ones achieved on NetMaker instances. The reason, again, is the correlation of the two first arc cost dimensions. In our encoding, the first arc cost dimension is the arc's length and the second one the duration needed to traverse the arc. According to Theorem 5 that also holds for the NAMOA_{dr}^{*}-lazy algorithm, paths are explored with increasing overall length. The correlation of these arc costs with the duration implies that the found minimal lex. solutions (minimal first cost dimension) also have a *very good* second cost dimension, that is, an almost minimal overall duration. Hence, regarding these two cost dimensions, the NAMOA_{dr}^{*}-lazy algorithm does not perform much more (erroneous) heap-insertions compared to the T-MDA algorithm. As a representative example, looking at the instances in the netM-25 group whose output contain between 5000 and 10 000 paths, the division of the average insertions of the T-MDA by the average insertions of the NAMOA_{dr}^{*}-lazy algorithm gives a ratio of 0.83. The same calculation on the COL instances whose output contain between 5000 and 10 000 paths gives a ratio of 0.97. Hence, on road instances the T-MDA is holding paths back before their insertion into the heap but almost every path that is held back ends up being inserted into the heap. Despite this fact, as shown in Figures 8–16, the superiority of the T-MDA on these instances is consistent.

7 | CONCLUSION

This paper introduces the *Targeted Multiobjective Dijkstra Algorithm* (T-MDA) for the One-to-One Multiobjective Shortest Path (MOSP) Problem. The T-MDA is a variant of the Multiobjective Dijkstra Algorithm (MDA) that guides the search towards the target node using a heuristic. In particular, this implies that the size of the priority queue used in the T-MDA is bounded by the number of nodes in the input graph, allowing efficient queue operations even on hard instances with exponential outputs. Keeping the practical performance of the algorithm in mind, explored paths that are not in the queue and cannot be discarded are stored in so called $\mathcal{NQP}$ lists. There is one $\mathcal{NQP}$ list for every arc in the graph and the $\mathcal{NQP}$ list for arc a stores the explored paths that are not in the queue and whose last arc is a . This arc based indexing allows the T-MDA to exploit the lexicographic sorting of paths in the algorithm's queue to maintain the $\mathcal{NQP}$ lists sorted using only constant time insertions and deletions at the front and back positions of the lists. In our extensive computational results on three dimensional MOSP instances, we compare the T-MDA with an enhanced version of the NAMOA_{dr}^{*} algorithm from the literature. The new version is called NAMOA_{dr}^{*}-lazy algorithm because it maintains explored paths in the algorithm's queue in a lazy way. This queue management was recently shown to be very efficient in the biobjective scenario and indeed our new NAMOA_{dr}^{*}-lazy algorithm clearly outperforms the NAMOA_{dr}^{*} from the literature. Our results show that this improved version is outperformed by the new T-MDA algorithm consistently. The T-MDA algorithm is even able to solve more hard instances than the NAMOA_{dr}^{*}-lazy algorithm. Its featured combination of a polynomially bounded queue size while still keeping other explored paths in lex ordered lists turns out to be very efficient to solve MOSP instances that require more than 2 s to be solved on a modern computer. Our implementations, instances, and results are publicly available on the GitHub repository mentioned in Section 6.

8 | FUTURE WORK

The T-MDA is designed to solve One-to-One MOSP instances with an arbitrary number of objectives. The experiments in our paper only consider instances with three objectives and large networks coming from real life applications (road networks). To further assess the benefits of the techniques used in the T-MDA compared to, for example, label correcting MOSP algorithms from the literature, extensive computational experiments on more than three objectives should be run. Since the number of efficient solution grows with increasing number of objectives, the used networks for these experiments will need to be artificial and smaller in size. The experiments from [16] with up to 10 objectives are a good starting point.

ACKNOWLEDGMENTS

We thank Ricardo Euler from Network Optimization department of the Zuse Institute Berlin for carefully proofreading the paper. His remarks contributed to improve the flow of the exposition. ZIB members conducted this work within the Research Campus MODAL—Mathematical Optimization and Data Analysis Laboratories, funded by the German Federal Ministry of Education and Research (BMBF) (fund number 05M20ZBM). Antonio Sedeno-Noda's work is partially founded by the Ministerio de Ciencia, Innovacion y Universidades of the Spanish Government (fund number PID2019-104928RB-I00). Open Access funding enabled and organized by Projekt DEAL.

DATA AVAILABILITY STATEMENT

Code and instances are available on <https://doi.org/10.5281/zenodo.7702018>.

ORCID

Pedro Maristany de las Casas  <https://orcid.org/0000-0002-4197-0893>

Antonio Sedeño-Noda  <https://orcid.org/0000-0003-0681-4585>

REFERENCES

- [1] S. Ahmadi, G. Tack, D. Harabor, and P. Kilby. Bi-objective search with bi-directional a^* , 29th annual European symposium on algorithms (ESA 2021), Vol. 204 of Leibniz international proceedings in informatics (LIPIcs), Schloss Dagstuhl—Leibniz-Zentrum für Informatik, Dagstuhl, Germany. 2021 3:1–3:15.
- [2] M. Baum, J. Dibbelt, D. Wagner, and T. Zündorf, *Modeling and engineering constrained shortest path algorithms for battery electric vehicles*, Transp. Sci. **54** (2020), 1571–1600.
- [3] M. Blanco, R. Borndörfer, and P. M. de las Casas. An a^* algorithm for flight planning based on idealized vertical profiles, 22nd symposium on algorithmic approaches for transportation modelling, optimization, and systems (ATMOS 2022), Vol. 106 of open access series in informatics (OASISs), Schloss Dagstuhl—Leibniz-Zentrum für Informatik, Dagstuhl, Germany. 2022 1:1–1:15.
- [4] F. K. Böckler, *Output-sensitive complexity of multiobjective combinatorial optimization with an application to the multiobjective shortest path problem*, Ph.D. thesis, Technische Universität Dortmund, Dortmund, 2018.
- [5] C. Demetrescu, A. Goldberg, and D. Johnson. 9th DIMACS Implementation Challenge—Shortest Paths. 2009 <http://www.diag.uniroma1.it/~sim;challenge9/>.
- [6] E. W. Dijkstra, *A note on two problems in connexion with graphs*, Numer. Math **1** (1959), 269–271.
- [7] M. Ehrgott, *Multicriteria optimization*, Springer-Verlag, Heidelberg, 2005.
- [8] A. V. Goldberg, H. Kaplan, and R. F. Werneck. Reach for A^* : Shortest path algorithms with preprocessing. The Shortest Path Problem 2006.
- [9] P. E. Hart, N. J. Nilsson, and B. Raphael, *A formal basis for the heuristic determination of minimum cost paths*, IEEE Trans. Syst. Sci. Cybern **4** (1968), 100–107.
- [10] L. Mandow and J. L. Pérez-de-la Cruz. A new approach to multiobjective a^* search, IJCAI. 2005.
- [11] L. Mandow and J. L. Pérez De La Cruz, *Multiobjective a^* search with consistent heuristics*, J. ACM **57** (2010), 1–25.
- [12] P. Maristany de las Casas. Targeted Multiobjective Dijkstra Algorithm: Initial Release. 2023 <https://doi.org/10.5281/zenodo.7702018>.
- [13] P. Maristany de las Casas, R. Borndörfer, L. Kraus, and A. Sedeño-Noda, *An FPTAS for dynamic multiobjective shortest path problems*, Algorithms **14** (2021), 43.
- [14] P. Maristany de las Casas, A. Sedeño-Noda, and R. Borndörfer, *An improved multiobjective shortest path algorithm*, Comput. Oper. Res. **135** (2021).
- [15] E. Q. V. Martins, *On a multicriteria shortest path problem*, Eur. J. Oper. Res. **16** (1984), 236–245.
- [16] J. M. Paixão and J. L. Santos, “Computational intelligence and decision making,” *Labeling methods for the general case of the multi-objective shortest path problem—A computational study*, Springer Netherlands, Dordrecht, 2013, pp. 489–502.
- [17] F. J. Pulido, L. Mandow, and J. L. de la Cruz, *Multiobjective shortest path problems with lexicographic goal-based preferences*, Eur. J. Oper. Res. **239** (2014), 89–101.
- [18] F. J. Pulido, L. Mandow, and J. L. P. de la Cruz, *Dimensionality reduction in multiobjective shortest path search*, Comput. Oper. Res. **64** (2015), 60–70.
- [19] A. Raith and M. Ehrgott, *A comparison of solution strategies for biobjective shortest path problems*, Comput. Oper. Res. **36** (2009), 1299–1331.

- [20] A. Raith, M. Schmidt, A. Schöbel, and L. Thom, *Extensions of labeling algorithms for multi-objective uncertain shortest path problems*, *Networks* **72** (2018), 84–127.
- [21] A. Sedeño-Noda and M. Colebrook, *A biobjective Dijkstra algorithm*, *Eur. J. Oper. Res.* **276** (2019), 106–118.
- [22] A. Skriver and K. Andersen, *A label correcting approach for solving bicriterion shortest-path problems*, *Comput. Oper. Res.* **27** (2000), 507–524.
- [23] C. H. Ulloa, W. Yeoh, J. A. Baier, H. Zhang, L. Suazo, and S. Koenig, *A simple and fast bi-objective search algorithm*. Proceedings of the 30th international conference on automated planning and scheduling (ICAPS), AAAI Press. 2020 143–151.

How to cite this article: P. Maristany de las Casas, L. Kraus, A. Sedeño-Noda, and R. Borndörfer, *Targeted multiobjective Dijkstra algorithm*, *Networks*. **82** (2023), 277–298. <https://doi.org/10.1002/net.22174>